Security Review

Cantina Managed

COINBASE: B20 V3

Cantina Managed review by:
Valerian Callens, Lead Security Researcher

CANTINA

October 2, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3
- 2.2 Codebase context . . . 3
- 2.3 Trust assumptions . . . 4
- 2.4 Cantina Managed Team Statement . . . 4

3 Findings **5**
- 3.1 Informational . . . 5
  - 3.1.1 Suggestions to improve the test coverage for the Transfer-executor policy . . . 5
  - 3.1.2 Suggestions to improve the test coverage for the current-token recipient checks 5
  - 3.1.3 Unknown policy IDs have inconsistent documented and actual behavior . . . 6
  - 3.1.4 B20 precompiles are missing from the Base documentation describing precompiles 8

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Sep 28th to Oct 1st the Cantina team conducted a review of base on commit hash f0a94cbf. The team identified a total of **4** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	4	4	0
Total	**4**	**4**	**0**

2.1 Scope

The security review had the following components in scope for base on commit hash f0a94cbf:

```
crates/common/precompiles/src
├── lib.rs
├── b20_asset
│   ├── dispatch.rs
│   ├── mod.rs
│   ├── versions.rs
│   └── logic
│       ├── interface.rs
│       ├── mod.rs
│       └── v3.rs
├── b20_stablecoin
│   ├── mod.rs
│   ├── versions.rs
│   └── logic
│       ├── mod.rs
│       └── v3.rs
├── common
│   ├── mod.rs
│   └── ops
│       ├── credit_recipient.rs
│       └── mod.rs
└── policy
    ├── dispatch.rs
    ├── mod.rs
    ├── versions.rs
    ├── abi
    │   ├── mod.rs
    │   └── v3.rs
    └── logic
        ├── interface.rs
        ├── mod.rs
        └── v3.rs
```

2.2 Codebase context

Base is an Ethereum Layer-2 rollup whose B-20 tokens and Policy Registry are implemented as native Rust EVM precompiles. This review covers the Denim (V3) execution paths and their fork/version routing

from Beryl through Denim, with the frozen V1/V2 implementations used as compatibility baselines.

The reviewed system comprises the Policy Registry, the B20 Asset and Stablecoin precompiles, their shared dispatch, ABI, storage, policy, token-accounting, and version-resolution layers.

Denim introduces three behavioral changes:

1. inverted Policy Registry IDs using bit 63 as a query-time NOT flag;
2. enforcement of `TRANSFER_EXECUTOR_POLICY` on all transfer variants;
3. rejection of balance credits to either the zero address or the current token address.

These changes apply across transfer, mint, batch-mint, and seize flows, while preserving the existing policy, allowance, role, pause, balance, event, and revert-order semantics.

The review also covers shared credit-recipient validation, policy-ID storage and evaluation, composite-policy behavior, ABI and selector changes, fork-gated dispatch, cross-feature interactions, and the associated colocated and golden tests. V1 and V2 production behavior is treated as frozen, the primary security scope is V3 logic and its correct activation only on Denim and later forks.

2.3 Trust assumptions

This review assumes that the underlying Base L2, EVM/revm execution engine, transaction admission, block production, consensus, and state-root verification operate according to their documented protocols. Their general correctness and availability are outside the scope of this review.

The review assumes that the configured hardfork is authoritative for precompile installation and version selection: the Policy Registry, Asset, and Stablecoin precompiles are unavailable before Beryl, use V1 at Beryl, V2 at Cobalt, and V3 at Denim and later. Version routing, ABI selection, and preservation of frozen V1/V2 behavior are nevertheless reviewed rather than trusted.

The Policy Registry singleton address and the B20 Factory, Asset, and Stablecoin precompile addresses are assumed to be installed according to the chain configuration. No trust is placed in callers of these precompiles: public EOAs and contracts may invoke all externally accessible paths, including factory initialization and view functions, without roles unless the implementation explicitly requires them.

The review assumes that the base-std specifications and the supplied Denim change guide define the intended external behavior, while the final merged Rust implementation is authoritative where the materials conflict. Frozen V1/V2 implementations and their golden tests are treated as compatibility baselines; no migration of existing token or Policy Registry storage is assumed.

2.4 Cantina Managed Team Statement

Base engaged Cantina to conduct a security review of its Denim B-20 V3 precompiles. We would like to thank the Base team for their responsiveness and constructive engagement throughout the review process. No significant issues were identified during the assessment, and the system is expected to operate as intended.

3 Findings

3.1 Informational

3.1.1 Suggestions to improve the test coverage for the Transfer-executor policy

Severity: Informational.

Context: v3.rs#L78-L101.

Description: The V3 transfer core checks policies in this order and compares the stored, raw IDs:

```
check_policy(policies.executor, caller)?;
if caller != from || policies.executor != policies.sender {
    check_policy(policies.sender, from)?;
}
check_policy(policies.receiver, to)?;
```

The following aspects are not covered by the tests:

1. Raw-ID comparison in self-delegated `transferFrom`:

The scenario should set `executor = baseId` and `sender = inverted(baseId)`, make the `caller` / `from` account authorized by `baseId`, and call `transferFrom` with `caller == from`. Because the IDs differ, the `sender` check must still run and reject the inverted `sender` policy. Repeat for Asset and Stablecoin, and assert unchanged balances, allowance, and logs. This would catch a future implementation that compares normalized IDs and skips the sender check.

2. `executor` / `sender` / `receiver` precedence:

Existing tests cover direct-transfer `sender` denial and isolated `executor` denial, while the unit test combines `sender` and `receiver` denial but does not assert which error wins. No test configures all three policies to deny simultaneously and asserts the first failure, and no test explicitly proves `receiver` precedence after `executor` and `sender` pass. The scenario should add one direct-transfer and one `transferFrom` case per token variant with distinct denying policies, expect `PolicyForbids(TransferExecutor)` first, then add the missing `sender`-only/`receiver`-only follow-ups where needed. For `transferFrom`, it should also assert the allowance is unchanged on every policy failure.

Recommendation: Add the two suggested test groups above for both `Asset` and `Stablecoin`.

Base: Fixed in PR 5464.

Cantina Managed: Fixed in the PR above by improving the test coverage.

3.1.2 Suggestions to improve the test coverage for the current-token recipient checks

Severity: Informational.

Context: v3.rs#L258-L259.

Description: The shared guard rejects the zero address and the current token address before allowance or policy evaluation:

```
B20CreditRecipient::new(to, token.token_address())
    .map_err(|_| InvalidReceiver { receiver: to })?;
```

The following aspects are not covered by the tests:

1. Memo selector coverage:

The three transfer/mint memo selectors currently have successful-path tests but no invalid-recipient tests. For both `Asset` and `Stablecoin`, a test scenario should add invalid current-token recipient tests for `transferWithMemo`, `transferFromWithMemo`, and `mintWithMemo`, but also add the equivalent zero-amount case to the already-covered `seizeWithMemo` rejection. A test scenario should also use a nonzero amount and at least one zero-amount call, assert `InvalidReceiver`, and no emission of `Transfer`, `Memo`, or `Seized` event.

2. Transfer policy precedence:

The existing tests use privileged dispatch or do not install the competing policy/allowance, so they do not cover the following combinations. For both token variants, a test scenario should call unprivileged `transfer(to = TOKEN)` with a denying `executor` or `receiver` policy and expect `InvalidReceiver`, not `PolicyForbids`. It should also have the same check for `transferFrom(to = TOKEN)` with a finite, insufficient allowance and a denying `executor`, expect `InvalidReceiver`, and assert that the allowance remains unchanged.

3. Mint and batch-mint policy precedence:

For both token variants, a test scenario should call privileged `mint(to = TOKEN)` and `mintWithMemo(to = TOKEN)` while `MintReceiver` denies the token, expect `InvalidReceiver` before policy evaluation and no state/events. For `Asset`s, a test scenario should also add a `batchMint` whose late recipient is `TOKEN` while `MintReceiver` denies it and assert `InvalidReceiver`, rollback of the earlier mint, unchanged supply, and no `Transfer` events.

4. Pause and role precedence:

A test scenario should add one representative test per token variant showing the documented ordering: a paused transfer/memo call to `TOKEN` returns `ContractPaused` because pause is checked before recipient validation, as well as an unprivileged mint or seizure to `TOKEN` without its role returns the role error because role checks precede recipient validation.

Recommendation: Add the suggested test groups above.

Base: Fixed in PR 5464.

Cantina Managed: Fixed in the PR above by improving the test coverage.

3.1.3 Unknown policy IDs have inconsistent documented and actual behavior

Severity: Informational.

Context: v3.rs#L531-L560.

Description: The specification states that an unknown or malformed policy ID must return `false`, for instance in `base-std/changelog/03_Denim_PolicyRegistry_not_policy.md`:

```
B20 stores a `uint64` policy ID per scope (`TRANSFER_FROM`, `TRANSFER_TO`,
↪  `MINT_RECEIVER`, `SEIZE_EXEMPT`, and other scopes) and calls
↪  `isAuthorized(policyId, account)` before gated operations. Invert is a
↪  query-time flip of that result, so it inherits the existing contract:
↪  `isAuthorized` never reverts, and a malformed or unknown ID returns `false`
↪  (deny).
```

Instead, the ABI interface documents empty-member semantics for unknown `ALLOWLIST` and `BLOCKLIST` IDs, without defining composite behavior, for instance in `base-std/src/interfaces/IPolicyRegistry.sol`:

```
/// @notice Returns whether `account` is authorized under `policyId`. Never reverts;
↪  unknown
```

```
///         or malformed IDs collapse to empty-member-set semantics (ALLOWLIST ->
↪  false,
///         BLOCKLIST -> true).
///
/// @dev Callers that store policy IDs MUST validate `policyExists(policyId)` at
↪  write time.
/// @dev Invert: `isAuthorized(invertedPolicyId(id), account)` returns the negated
///      result of the base. Applies to every policy type.
///
/// @param policyId Policy to query.
/// @param account  Account to check.
///
/// @return Whether `account` is authorized.
function isAuthorized(uint64 policyId, address account) external view returns
↪  (bool);
```

The implementation of `is_authorized()` in `common/precompiles/src/policy/logic/v3.rs`:

```
// Malformed IDs (type byte > INTERSECT) are treated as unauthorized rather than
↪  reverting.
if !Self::is_well_formed(policy_id) {
    return Ok(false);
}
match Self::policy_id_type(policy_id) {
    Self::UNION_TYPE => self.is_authorized_union(storage, policy_id, account),
    Self::INTERSECT_TYPE => self.is_authorized_intersect(storage, policy_id,
    ↪  account),
    Self::ALLOWLIST_TYPE => storage.read_member(policy_id, account),
    Self::BLOCKLIST_TYPE => Ok(!storage.read_member(policy_id, account)?),
    _ => unreachable!("is_well_formed rejects type bytes > INTERSECT"),
}
```

behaves as follows:

```
malformed ID          -> false
missing ALLOWLIST     -> false
missing BLOCKLIST     -> true
missing UNION         -> false
missing INTERSECT     -> true
missing inverted base -> false
```

The concrete fail-open case is a missing but well-formed `INTERSECT` ID:

```
missingIntersect = (INTERSECT << 56) | 999

// well-formed ID: dispatch by type, without policy_exists()
INTERSECT_TYPE => is_authorized_intersect(storage, policy_id, account)

// no child record: empty loop returns the identity value, true
for child in read_children(policy_id)? {
    if !is_authorized(child, account)? { return Ok(false); }
}
Ok(true)
```

Therefore:

```
policyExists(missingIntersect) == false
isAuthorized(missingIntersect, account) == true
```

Code of `is_authorized_intersect()`:

```rust
/// Evaluates an INTERSECT (AND) composite over its live child set: authorized only
↪  if every
/// child authorizes. An empty set is authorized.
fn is_authorized_intersect<S: PolicyAccounting>(
    &self,
    storage: &S,
    policy_id: u64,
    account: Address,
) -> Result<bool> {
    for child in storage.read_children(policy_id)? {
        if !self.is_authorized(storage, child, account)? {
            return Ok(false);
        }
    }
    Ok(true)
}
```

This is a public `PolicyRegistry` view inconsistency, not currently a B20 token configuration bypass: in `Asset` and `Stablecoin`, `updatePolicy()` rejects the same ID because `policyExists` is false. An integration that treats `isAuthorized == true` as sufficient could nevertheless accept authorization from a nonexistent policy.

The mismatch is explicit: the specification's blanket fail-closed wording implies `false` for every unknown ID, while the non-inverted `BLOCKLIST` and `INTERSECT` paths return `true`.

Recommendation: Add a characterization test for the full matrix above, especially `policyExists(missingIntersect) == false` paired with `isAuthorized(missingIntersect, account) == true`. Then either document the matrix explicitly in both specifications and ABI comments, or change both V2 and V3 through an intentional versioned behavior change so every missing policy fails closed.

Base: Fixed in PR 234 (`base-std` repository) and in PR 5457 (`base` repository).

Cantina Managed: Behavior confirmed as intended, clarified by updating the specs in `base-std` and covered by tests in the `base` repository, file `b20_policy_v3_golden.rs`.

3.1.4 B20 precompiles are missing from the Base documentation describing precompiles

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Description: The B20 precompiles are not included in the Base documentation describing precompiles: <precompiles>.

Recommendation: Consider adding the B20 precompiles to the Base precompile documentation.

Base: Fixed here.

Cantina Managed: Fixed, the B20 precompiles are now included in the Base documentation describing precompiles.